

Short technical report

Official template: general information, feature checklist, architecture, empirical evidence and challenges.

1. GENERAL INFORMATION & DELIVERABLE LINKS

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 3 - OCR Expense Tracker & Transfer Parser

Student: Dương Bảo Đạt | **Student ID:** 23IT046

Team: Individual | **Role:** Full-stack mobile developer | **Contribution:** 100%

Submission date: 07/10/2026

GitHub: <https://github.com/duongdatdev/transfer-lens>

APK + video: [GitHub Release v1.0.0](#) - download all deliverables

PDF: [transfer-lens-report.pdf](#) (this report)

Adapted scenario: Instead of paper receipts, the user imports a completed bank-transfer image. The parser extracts amount, date, sender/recipient and description. A keyword rule suggests a category; unknown descriptions require manual selection. This scope change follows the student request and is documented for instructor review.

2. FEATURE IMPLEMENTATION CHECKLIST

Required feature	Status	Implementation / acceptance
Image input + crop	Implemented	Gallery screenshot import, native crop/rotate; optional camera with flash, focus and frame.
On-device OCR	Validated	Bundled ML Kit Latin model; native Android OCR exercised on fictional image assets.
Regex heuristics	Validated	VND, calendar date, explicit parties, description/reference; ambiguity and failure warnings.
Category + review	Validated	Five mapped categories plus Other; editable fields, confirmation and duplicate warning.
Provider + SQLite CRUD	Validated	Local records survive database reopen; full-size images and resized thumbnails cached privately.
Canvas visualizations	Validated	Animated, interactive CustomPainter donut and weekly bars; no chart package.
Material 3 + accessibility	Validated	Light/dark/system, responsive layout, large-text and reduced-motion widget checks.
Release + report	Prepared	Signed APK, native walkthrough video, four-page report; online publication status shown above.

Architecture & validation

A modular Flutter application with local data ownership and transparent parsing rules.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE



Approved review → Provider ExpenseStore → SQLite repository + private image storage → history + CustomPainter charts.

The app never initiates a transfer. Direction defaults to expense and remains editable. Income is recorded separately and excluded from spending charts.

Directory	Responsibility
lib/domain	Immutable transaction model; integer VND normalization; label/date/status rules; accent-insensitive description keywords.
lib/data + lib/state	Repository abstraction, schema constraints/indexes, parameterized CRUD, Provider state, duplicate warning and persistent theme.
lib/services	Native recognizer lifetime and visual reading order; private original images; thumbnail resizing in a background isolate.
lib/ui + widgets	Import, camera, review, history and settings; interactive donut/bar painters with reduced-motion support.
test + integration_test	Parser edge cases, real SQLite reopen, adaptive UI, native OCR and end-to-end review/cache/delete flow.

Failure handling: Missing/ambiguous fields remain editable. Database writes happen after review and image caching. If a write fails, new cache files are cleaned up. Delete the row before cleaning its images. Camera lifecycle and OCR resources are disposed. Android backups are disabled; local files are not additionally encrypted.

Measured verification

Toolchain: Flutter 3.47.5, Dart 3.13.4, Android SDK 36, Java 21. Android API 24+ is the release target. iOS 15.5+ is configured but untested on Windows.

Checks: Formatting and analyzer; 31 unit/widget tests; native Android review/cache/CRUD integration. A signed optimized release also recognized the sample without Internet permission (2,595 ms).

OCR cold run: 4,488 ms on the emulator in the initial integration test. Further demo observations are below. These are observations, not a physical-device performance claim.

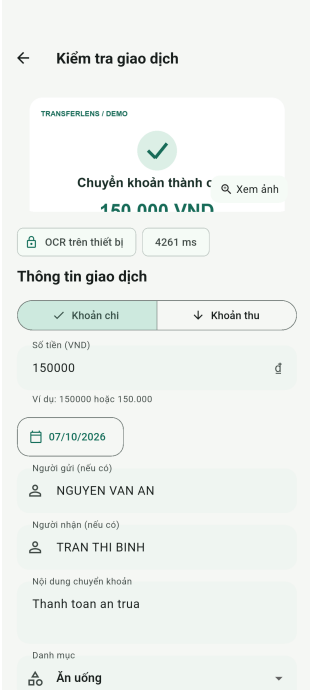
DEMO-OTHER-003: 843 ms
DEMO-STUDY-002: 1468 ms
DEMO-FOOD-001: 4261 ms

Empirical evidence

Actual screenshots from the Android emulator; the banking confirmations contain fictional demo data.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

A. Native OCR review



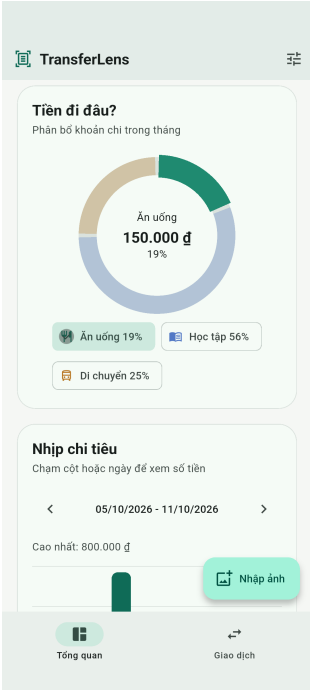
The imported image, measured OCR duration and editable amount appear before saving.

B. Manual classification



A generic transfer description has no confident keyword match, so a category must be selected.

C. Canvas analytics



Saved expenses feed the category donut. Selected categories expose exact VND amounts.

D. Persistent history



Searchable records with thumbnails, category/date labels and amounts; Material 3 dark mode.

Challenges & resolutions

Design decisions, validation limits and the practical submission handoff.

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1 - Financial numbers and banking layouts.

A confirmation can contain a balance, fee, reference and account number, all near the real amount. The parser scores explicit amount labels above currency-only values and excludes unrelated fields. It validates calendar dates, treats conflicting candidates as unresolved, and reads only explicitly labeled parties. Names and missing data remain editable. Classification uses only description keywords with phrase boundaries, avoiding accidental matches inside names or unrelated words.

Challenge 2 - Native release build and adaptive UI.

Windows Kotlin caches failed across different drives; incremental compilation was disabled. R8 required narrowly scoped rules for unused ML Kit scripts and explicit preservation of reflectively created component-registrar constructors. A signed release smoke test confirmed OCR without Internet permission after the fix. Large-text tests found overflowing summary labels and chart centers; flexible text layout resolved the failures.

Acceptance scope and remaining checks

Validated: Android native OCR on fictional confirmations, reviewed saving, SQLite persistence, image/thumbnail lifecycle, category fallback, duplicate detection, canvas selection, responsive light/dark layouts and release signing.

Physical-device follow-up: Verify camera flash/focus, photo-picker/crop flow, bank-specific image accuracy and latency on the target phone. Emulator OCR measurements do not establish the assignment's sub-100ms target.

iOS: Source permissions, Podfile and deployment target are prepared; a macOS/Xcode device build is still required.

Privacy: No account, cloud OCR or bank integration. Images, raw text and transactions remain local. This app is a personal record and does not prove that money was transferred.

Build and demonstrate

Setup: flutter pub get; flutter run -d <android-device>.

Quality: flutter analyze; flutter test; flutter test integration_test/app_test.dart -d <android-device>.

Signed release: Set private android/key.properties, then flutter build apk --release.

Video: Actual Flutter screens and real ML Kit processing, automated using integration_test with fictional data. The source includes a reproducible walkthrough and recording script.

References

Google ML Kit: bundled Android text-recognition model

Flutter ML Kit plugin: mobile platforms and model setup

Flutter CustomPainter: canvas rendering API

Image Cropper: native configuration

Original report structure: supplied Mini-Project-3-Report-Template.md